

Anmerkungen zum Lean-Code

Cut-and-Project-Periodenformel

Review-Kommentare

1. Mai 2026

Gesamturteil

Der Lean-Code ist beeindruckend und formalisiert einen erheblichen Teil der kombinatorischen Kernstruktur des Papers. Besonders stark sind die Formalisierung der Restklassenverteilung, der zyklischen Intervalle, der Minimalitätsargumente und der set-valued Dichotomie.

Gleichzeitig sollte der Code in seiner jetzigen Form nicht ohne Präzisierung als vollständige Formalisierung der geometrischen Cut-and-Project-Konstruktion präsentiert werden. Der wichtigste Punkt ist die Modelltreue: Der Code formalisiert im Wesentlichen ein normalisiertes Restklassenmodell, während die tatsächlichen geometrischen Restklassen im Paper durch

$$c_r \equiv -\alpha\beta^{-1}r \pmod{D}$$

gegeben sind.

1 Stärken des Codes

Die folgenden Bestandteile sind mathematisch und formal sehr gelungen:

- `coprime_alpha_D` und `coprime_beta_D` bilden die elementare Zahlentheorie sauber ab.
- `count_hits` ist eine gute Formalisierung der Trefferanzahl in einem Block aufeinanderfolgender Restklassen.
- `count_hits_lt_D`, `count_hits_ge_D`, `uniform_residue_distribution` und `non_uniform_residue_distribution` treffen genau die zentrale Fallunterscheidung.
- `cyclic_interval_stabilizer_trivial` formalisiert den entscheidenden Minimalitätsschritt.
- `generic_minimality` ist konzeptionell überzeugend: Aus einem Periodenkandidaten entsteht eine Restklassen-Translation; diese stabilisiert die schwere Menge; der triviale Stabilisator erzwingt die behauptete Minimalität.
- Die set-valued Konstruktion über `set_indicator`, `set_size`, `set_sorted` und `set_difference_sequence` spiegelt die mathematische Idee klar wider.

Der Code ist damit keine bloße Skizze, sondern eine ernsthafte formale Begleitung des mathematischen Arguments.

2 Wichtigster kritischer Punkt: Der Multiplikator wird nicht durchgezogen

Im Paper sind die tatsächlichen projektiven Restklassen

$$c_r \equiv -\alpha\beta^{-1}r \pmod{D}.$$

Im Lean-Code werden zwar

```
def multipliiier ...
def residue_bijection ...
```

definiert, aber der Hauptteil arbeitet später mit

```
def count_hits (D : Nat) [NeZero D] (r0 N : Nat) (x : ZMod D) : Nat :=
  (Finset.range N).filter (fun i => (r0 + i : ZMod D) = x) |>.card
```

Das entspricht den Restklassen

$$r_0, r_0 + 1, \dots, r_0 + N - 1 \pmod{D},$$

nicht unmittelbar den geometrischen Restklassen

$$mr_0, m(r_0 + 1), \dots, m(r_0 + N - 1) \pmod{D}, \quad m = -\alpha\beta^{-1}.$$

Mathematisch ist das für die Periodenlänge wahrscheinlich unschädlich, weil Multiplikation mit einer Einheit modulo D nur eine Bijektion der Restklassen ist. Aber die tatsächlich sortierte Gap-Folge kann durch diese Multiplikation anders aussehen. Daher sollte entweder der Multiplikator in der konkreten Enumeration formal eingebaut werden, oder die Lean-Sektion im Paper sollte klar sagen, dass ein normalisiertes kombinatorisches Restklassenmodell formalisiert wurde.

3 Vorschlag: count_hits_unit

Eine mögliche Verbesserung wäre eine allgemeinere Trefferfunktion:

```
def count_hits_unit
  (D : Nat) [NeZero D]
  (u : (ZMod D)^*)
  (r0 N : Nat)
  (x : ZMod D) : Nat :=
  (Finset.range N).filter
    (fun i => u.val * (r0 + i : ZMod D) = x) |>.card
```

Für das geometrische Modell wäre dann

```
u = multipliiier alpha beta h
```

und die tatsächliche Restklassenabbildung wäre formal enthalten.

Dann sollte man ein Transferlemma beweisen:

```
count_hits_unit D u r0 N x =
count_hits D r0 N (u^{-1}.val * x)
```

Damit könnten viele vorhandene Beweise weiterverwendet werden. Der schwere Satz wäre dann nicht mehr direkt in der natürlichen Restklassenordnung ein Intervall, sondern nach Rücktransport durch u^{-1} . Das entspricht genau der Argumentation im Paper.

4 Benennung und Reichweite der Formalisierung

Der Name

```
main_theorem_concrete
```

wirkt etwas stärker, als die aktuelle Modellierung rechtfertigt. Er klingt so, als sei die volle geometrische Konstruktion formalisiert. Treffender wären Namen wie:

```
main_theorem_residue_model  
main_theorem_normalized_residue_model
```

Ähnliches gilt für die Typeclass

```
class GeometricProjection ...
```

Die Axiome sind eher periodisch-kombinatorisch als geometrisch. Ein präziserer Name wäre zum Beispiel:

```
class ResidueProjectionModel ...  
class PeriodicResidueModel ...
```

Wenn der Name `GeometricProjection` beibehalten wird, sollte der Kommentar deutlicher sein, etwa:

```
--  
Abstract interface for the combinatorial residue model obtained after  
reducing the geometric cut-and-project construction.  
-/
```

5 Umbenennung von `non_uniform_residue_distribution`

Das Theorem

```
theorem non_uniform_residue_distribution ...
```

gilt offenbar auch im Fall $s = 0$, also im uniformen Fall $D \mid N$. Denn dann ist die Anzahl der $q + 1$ -Klassen gleich 0, und die Anzahl der q -Klassen gleich D .

Der Name ist daher etwas irreführend. Besser wäre:

```
theorem residue_distribution  
theorem quotient_remainder_residue_distribution
```

Daraus könnten dann zwei Spezialisierungen abgeleitet werden:

```
theorem non_uniform_residue_distribution_of_not_dvd ...  
theorem uniform_residue_distribution ...
```

6 Der degenerierte Fall ist im Code stärker als im Paper

Der Code beweist in

```
lemma period_degenerate_concrete ...
```

tatsächlich die Minimalität der Periode N/D . Das ist sehr gut. Im Paper sollte der analoge Beweisschritt ebenfalls explizit ergänzt werden: Die Folge

$$\underbrace{0, \dots, 0}_{N/D-1}, 1, \underbrace{0, \dots, 0}_{N/D-1}, 1, \dots$$

hat nicht nur Periode N/D , sondern minimale Periode N/D . Für $N/D > 1$ liegt die 1 genau einmal pro Block der Länge N/D , sodass keine kleinere positive Periode möglich ist.

7 Architektur von main_theorem

Das Theorem

```
theorem main_theorem ...  
  [GeometricProjection alpha beta omega (difference_sequence alpha beta omega)] :
```

ist eine Mischform: Es verwendet abstrakte Axiome, aber bereits die konkrete Sequenz `difference_sequence`. Architektonisch sauberer wäre eine abstrakte Version:

```
theorem main_theorem_abstract  
  (seq : Z -> Z)  
  [GeometricProjection alpha beta omega seq] :  
  ...  
  HasPeriodLength seq L
```

und anschließend ein konkreter Wrapper:

```
theorem main_theorem_concrete :  
  HasPeriodLength (difference_sequence alpha beta omega) L
```

Das ist kein mathematischer Fehler, aber es würde die Struktur des Codes deutlicher machen.

8 Modulo durch set_size

Die Definition

```
def set_sorted ... :=  
  let M := (set_size alpha beta omega : Z)  
  let r := (i % M).toNat  
  ...
```

verwendet Modulo durch `set_size`. Lean ist total und erlaubt diese Definition auch bei $M = 0$. In den relevanten Fällen wird später bewiesen, dass `set_size = N` oder `set_size = D` gilt und damit positiv ist. Mathematisch ist das also in den Theoremen kontrolliert.

Für bessere Lesbarkeit und Robustheit wäre aber ein Lemma sinnvoll:

```
lemma set_size_pos ...
```

Dieses Lemma könnte explizit dokumentieren, dass die set-valued Enumeration in den relevanten Fällen wohldefiniert als Periodenblock der positiven Länge ist.

9 Heartbeat-lastige Beweise

Mehrere Beweise verwenden hohe Heartbeat-Grenzen, zum Beispiel:

```
set_option maxHeartbeats 400000  
set_option maxHeartbeats 800000
```

Das ist für ein Forschungsprojekt akzeptabel, aber für langfristige Wartbarkeit fragil. Besonders `sorted_multiset_add_q` wirkt technisch schwer. Es wäre besser, die dort verwendeten Aussagen über Division, Modulo und den Fall $N = qD$ in kleinere allgemeine Lemmas auszulagern.

10 Set-valued Teil

Der set-valued Teil ist sehr überzeugend strukturiert:

- Phase A: strukturelle Dichotomie über `count_hits_lt_D` und `count_hits_ge_D`.
- Phase B: abstrakter Satz `set_main_theorem`.
- Phase C: konkrete Konstruktion über `set_indicator`, `set_cumulative_hits`, `set_V`, `set_size`, `set_sorted` und `set_difference_sequence`.

Das ist didaktisch klar und mathematisch passend: Multiplizitäten werden auf $\{0, 1\}$ reduziert.

11 Formulierungsvorschlag für das Paper

Mit dem Code in der jetzigen Form wäre folgende Beschreibung angemessen:

The Lean formalisation verifies the algebraic and combinatorial core of the argument in a normalised residue model. It formalises the coprimality lemmas, the uniform and non-uniform residue-distribution statements, the cyclic-interval stabiliser argument, the generic minimality proof, the degenerate case, and the set-valued dichotomy. The Euclidean cut-and-project geometry is not formalised from first principles; instead, the formalisation starts from the residue model obtained after reducing the geometry to arithmetic progressions modulo D .

Falls der Multiplikator $m = -\alpha\beta^{-1}$ in die konkrete Enumeration eingebaut wird, kann diese Formulierung entsprechend stärker ausfallen.

Schlussurteil

Der Lean-Code ist sehr gut als kombinatorische Formalisierung. Er enthält die entscheidenden Minimalitätsargumente und stützt das Paper substanziell.

Der zentrale Vorbehalt lautet:

Der Code formalisiert aktuell nicht sichtbar die tatsächliche geometrische Restklassenabbildung

$$r \mapsto -\alpha\beta^{-1}r \pmod{D},$$

sondern arbeitet im Wesentlichen mit dem normalisierten Fall $m = 1$.

Empfehlung:

1. Für arXiv kurzfristig: Die Lean-Sektion vorsichtig formulieren und von einem normalisierten Restklassenmodell sprechen.
2. Für maximale Qualität: `count_hits_unit` einführen und den Multiplikator m tatsächlich durch die konkrete Enumeration ziehen.
3. Für Codepflege: `non_uniform_residue_distribution` umbenennen, `main_theorem` abstrakter machen und heartbeat-lastige Beweise modularisieren.

Dann wäre die Lean-Formalisation eine sehr solide und glaubwürdige Begleitung des Papers.